

Lista de Código: API com Express

Turma: LionsDev

Tópicos: `express()`, `express.json()`, rotas `GET` / `POST` / `PUT` / `DELETE`, `req.body`, `req.params`, `res.status()`, `res.send()` / `res.json()` e status codes (200, 201, 400, 404).

Nesta lista você escreve as rotas de uma API. Base do arquivo: `import express from "express"; const app = express(); app.use(express.json()); app.listen(3000);`. Teste cada rota no Insomnia ou no Postman.

Parte 0 — Treino rápido (aquecimento)

Uma rota por vez. Escreva, suba o servidor, teste.

1. Crie o app Express e faça ele escutar na porta `3000`.
2. Ative o middleware que lê JSON do corpo da requisição (`express.json()`).
3. Crie uma rota `GET /` que responde `"API no ar!"`.
4. Crie uma rota `GET /ping` que responde o JSON `{ mensagem: "pong" }`.
5. Numa rota `POST`, leia `req.body` e imprima no console.
6. Numa rota `GET /produtos/:id`, leia e imprima `req.params.id`.
7. Responda com status 201 e um objeto (`res.status(201).send(...)`).
8. Responda com status 404 e a mensagem `{ erro: "Não encontrado" }`.
9. Responda com status 400 quando faltar um campo obrigatório.
10. Converta `req.params.id` (string) para número com `Number()` ou `parseInt()`.

Parte 1 — Complete o código

1. Rota de listagem

Complete a rota que devolve todos os produtos.

```
const produtos = [{ id: 1, nome: "Mouse" }];

app.get("/produtos", (req, res) => {
```

```
// TODO: responda status 200 com o array 'produtos'
});
```

2. Rota de criação

Complete a rota **POST** lendo o corpo e validando.

```
app.post("/produtos", (req, res) => {
  const { nome, preco } = req.body;

  if (/* TODO: nome OU preco não enviados */) {
    return res.status(400).send({ erro: "nome e preco são obrigatórios" });
  }

  const novo = { id: produtos.length + 1, nome, preco };
  produtos.push(novo);
  // TODO: responda status 201 com o novo produto
});
```

3. Rota por ID

Complete a busca por parâmetro de rota.

```
app.get("/produtos/:id", (req, res) => {
  const id = Number(req.params.id);
  const produto = produtos.find((p) => p.id === id);

  if (!produto) {
    // TODO: responda 404 com mensagem
  }
  // TODO: responda 200 com o produto
});
```

Parte 2 — Ache o bug

4. req.body vazio

No **POST**, **req.body** sempre chega **undefined**. Falta uma linha na configuração do app. Qual?

```
const app = express();
// ??? -> faltou algo aqui
```

```
app.post("/itens", (req, res) => {  
  console.log(req.body); // undefined  
  res.send("ok");  
});
```

5. Sem status de erro

Esta rota responde **200** mesmo quando não acha o item, e ainda tenta responder duas vezes. Conserte usando **return**.

```
app.get("/itens/:id", (req, res) => {  
  const item = itens.find((i) => i.id === Number(req.params.id));  
  if (!item) {  
    res.send({ erro: "não achou" }); // BUG: falta status e falta return  
  }  
  res.send(item);  
});
```

Parte 3 — Prever o comportamento

6. Qual a resposta?

Dada a rota abaixo e o array **usuarios = [{ id: 1, nome: "Ana" }]**, diga status e corpo da resposta para cada requisição.

```
app.get("/usuarios/:id", (req, res) => {  
  const u = usuarios.find((x) => x.id === Number(req.params.id));  
  if (!u) return res.status(404).send({ erro: "não encontrado" });  
  res.status(200).send(u);  
});
```

- (a) GET /usuarios/1
- (b) GET /usuarios/9

Parte 4 — Escreva do zero

7. API de Tarefas (Desafio)

Monte uma API REST completa em memória (array `tarefas`, cada uma `{ id, titulo, concluida }`), com as rotas:

Método	Rota	O que faz	Status sucesso
GET	<code>/tarefas</code>	lista todas	200
GET	<code>/tarefas/:id</code>	uma por id (404 se não achar)	200
POST	<code>/tarefas</code>	cria (valida <code>titulo</code> ; 400 se faltar)	201
PUT	<code>/tarefas/:id</code>	atualiza (404 se não achar)	200
DELETE	<code>/tarefas/:id</code>	remove (404 se não achar)	200

Teste cada rota no seu cliente HTTP e confira os status codes.

Dica: sem `express.json()` o `req.body` vem `undefined`, então ele é obrigatório pra ler o corpo da requisição. O `req.params` chega sempre como string, converta o id com `Number()`. E use `return res.status(...)` pra não cair no erro de responder duas vezes na mesma requisição.

LionsDev • Professor Nicolas Cardoso Motta

Lista de Código: API com Express - Módulo 07